

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

NIVEL 2 (INTERMEDIO)

Curso 2: Algoritmos Avanzados y Estructuras de Datos

«Optimización de Memoria, Notación Big-O, Pilas, Colas, Búsqueda Binaria y Programación Dinámica»

Manual completo del curso compilado oficialmente. Incluye fundamentación teórica, metáforas conceptuales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor Principal: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Total de Módulos / Clases: 3 | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura del Manual de Curso

Este libro unificado contiene el desglose secuencial de cada módulo formativo, estructurado con rigor de ingeniería y diseñado para servir como manual de referencia permanente.

Plan de Estudios y Hoja de Ruta

El curso está dividido en las siguientes unidades temáticas secuenciales:

Código	Unidad y Metáfora	Objetivo Principal
Módulo 01	Módulo 01: Estructuras de Datos Avanzadas «Pilas LIFO, Colas FIFO y Árboles Jerárquicos»	Comprender las disciplinas de acceso LIFO y FIFO y el coste temporal $O(1)$ vs $O(n)$...
Módulo 02	Módulo 02: Ordenamiento, Búsqueda y Big-O «El Diccionario por la Mitad y Divide y Vencerás»	Comprender cómo escala el tiempo de ejecución a medida que el tamaño de entrada ...
Módulo 03	Módulo 03: Recursividad y Programación Dinámica «Las Muñecas Rusas (Matrioshkas) y la Libreta de Apuntes»	Comprender la descomposición recursiva y cómo la memoización transforma compleji...



Criterio de Progresión

Cada unidad construye sobre los cimientos de la anterior. Se recomienda dominar la teoría, analizar el diagrama de arquitectura y replicar el código de cada sección antes de avanzar a la siguiente.

Módulo 01: Estructuras de Datos Avanzadas

Las listas básicas no siempre son la estructura óptima cuando la velocidad de inserción y extracción en los extremos es crítica.

☀ Metáfora Central: Pilas LIFO, Colas FIFO y Árboles Jerárquicos

Una Pila (Stack) es como una pila de platos: el último que colocas arriba es el primero que lavas (LIFO: Last In, First Out). Una Cola (Queue) es como la fila del supermercado: el primero que llega es el primero en ser atendido (FIFO: First In, First Out).

Principios Teóricos y Modelo Mental

`collections.deque` en Python permite inserciones y extracciones $O(1)$ tanto por la izquierda como por la derecha, a diferencia de `list.pop(0)` que cuesta $O(n)$.

Los conjuntos (sets) implementan álgebra de conjuntos (unión, intersección, diferencia) con consultas $O(1)$ y garantizan elementos únicos.

⚡ Regla de Oro en Python

Para colas FIFO de alto rendimiento en Python, utiliza siempre `collections.deque` en lugar de listas estándar.

Comparativa Visual: Pila (LIFO) vs Cola (FIFO)

Mecanismos de inserción (push/enqueue) y extracción (pop/dequeue) en memoria:



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Operación Push / Enqueue: Ingreso de nuevo elemento.	Elemento en memoria
2. Evaluación	En Stack (LIFO): Se coloca en el tope y se extrae del tope.	Último en entrar = Primero en salir
3. Transformación	En Queue (FIFO): Se ingresa por la cola y se extrae por la cabeza.	Primero en entrar = Primero en salir
4. Retorno / Salida	Árboles: Ramifican decisiones jerárquicas izquierda/derecha.	Acceso logarítmico $O(\log n)$



Visualización Mental

Las pilas gestionan llamadas de funciones y el botón Deshacer (Ctrl+Z); las colas gestionan mensajes y colas de impresión.

Validador de Paréntesis Balanceados con Pilas

main.py (Python 3.10+)

PEP 8 Compliant

```
def validar_parentesis(expresion: str) -> bool:
    pila: list[str] = []
    pares = {"(": ")", "{": "}", "[": "]"

    for char in expresion:
        if char in pares.values():
            pila.append(char) # Push
        elif char in pares:
            if not pila or pila.pop() != pares[char]: # Pop & Check
                return False

    return len(pila) == 0

# Pruebas
print(validar_parentesis("{[()()]}")) # True
print(validar_parentesis("{[()]}" )) # False
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar `list.pop(0)` para implementar una cola; obliga a desplazar todos los elementos restantes en memoria generando complejidad $O(n)$.

✗ Antipatrón

```
cola = []
cola.insert(0, item) #  $O(n)$  en cada inserción
```

✓ Patrón Correcto

```
from collections import deque
cola = deque()
cola.append(item) #  $O(1)$  instantáneo
```

Módulo 02: Ordenamiento, Búsqueda y Big-O

En software la velocidad no se mide en segundos, sino en cómo crece el número de operaciones en función del volumen de datos (n).

☀ Metáfora Central: El Diccionario por la Mitad y Divide y Vencerás

Si buscas una palabra en un diccionario de 1,000 páginas hojeando página por página (búsqueda lineal), puedes tardar 1,000 pasos. Si abres el diccionario por la mitad exacta y descartas la mitad irrelevante (búsqueda binaria), encontrarás la palabra en solo 10 pasos.

Principios Teóricos y Modelo Mental

Escalas de Complejidad: $O(1)$ Constante < $O(\log n)$ Logarítmica < $O(n)$ Lineal < $O(n \log n)$ Casi-lineal < $O(n^2)$ Cuadrática.

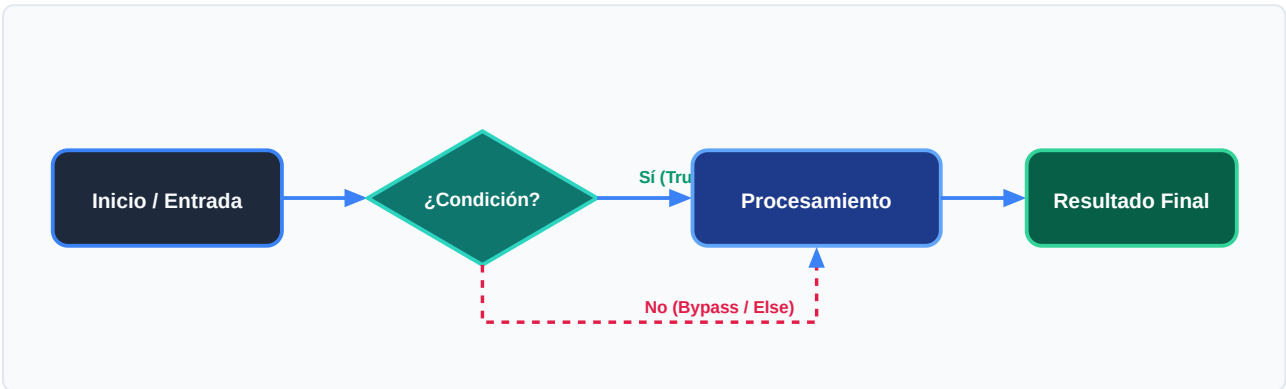
Búsqueda Binaria requiere que la colección esté previamente ordenada para garantizar reducción del espacio de búsqueda a la mitad.

⚡ Regla de Oro en Python

Evita los bucles anidados innecesarios: dos bucles anidados sobre n elementos convierten un algoritmo de $O(n)$ a $O(n^2)$.

Diagrama de Búsqueda Binaria (Divide & Conquer)

Estrategia de reducción logarítmica del intervalo de búsqueda con punteros low, mid y high.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Calcula el punto medio: $mid = (low + high) // 2$.	$low=0$, $high=n-1$, mid
2. Evaluación	Compara el elemento en 'mid' con el objetivo buscado.	Evalúa igualdad
3. Transformación	Si $objetivo < array[mid]$, descarta la mitad derecha ajustando $high = mid - 1$.	Espacio reducido al 50%
4. Retorno / Salida	Si $objetivo > array[mid]$, descarta la mitad izquierda ajustando $low = mid + 1$.	Repite hasta converger



Visualización Mental

La búsqueda binaria puede encontrar un registro entre 4 mil millones de elementos en tan solo 32 comparaciones.

Búsqueda Binaria y QuickSort en Python

main.py (Python 3.10+)

PEP 8 Compliant

```
def busqueda_binaria(lista: list[int], objetivo: int) -> int:
    low, high = 0, len(lista) - 1
    while low <= high:
        mid = (low + high) // 2
        if lista[mid] == objetivo:
            return mid # Encontrado
        elif lista[mid] < objetivo:
            low = mid + 1
        else:
            high = mid - 1
    return -1 # No existe

def quicksort(arr: list[int]) -> list[int]:
    if len(arr) <= 1:
        return arr
    pivote = arr[len(arr) // 2]
    izq = [x for x in arr if x < pivote]
    centro = [x for x in arr if x == pivote]
    der = [x for x in arr if x > pivote]
    return quicksort(izq) + centro + quicksort(der)
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Ejecutar búsqueda binaria sobre una lista no ordenada; produce falsos negativos y resultados erráticos.

✗ Antipatrón

```
desordenada = [9, 1, 5, 2]
busqueda_binaria(desordenada, 5) # ¡Falla!
```

✓ Patrón Correcto

```
ordenada = sorted(desordenada)
busqueda_binaria(ordenada, 5) # Retorna índice correcto
```

Módulo 03: Recursividad y Programación Dinámica

La recursividad ocurre cuando una función se invoca a sí misma para resolver una versión más pequeña del mismo problema.

☀ Metáfora Central: Las Muñecas Rusas (Matrioshkas) y la Libreta de Apuntes

La recursión es como abrir una muñeca rusa (Matrioshka): abres una y hay otra idéntica más pequeña dentro, hasta llegar a la más diminuta que no se puede abrir (el Caso Base). La memoización es como tener una libreta de apuntes: cuando resuelves un cálculo difícil, anotas el resultado para no tener que volver a calcularlo jamás.

Principios Teóricos y Modelo Mental

Todo algoritmo recursivo DEBE tener al menos un Caso Base para detener las llamadas antes de saturar el Call Stack (RecursionError).

Programación Dinámica (DP): Técnica para resolver problemas complejos descomponiéndolos en subproblemas y guardando sus soluciones.

⚡ Regla de Oro en Python

Sin memoización, Fibonacci recursivo tiene complejidad $O(2^n)$; con memoización se reduce a $O(n)$.

Árbol de Llamadas Recursivas vs Tabla de Memoización

Eliminación de ramas redundantes en el árbol de ejecución mediante caché en memoria.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Llamada inicial a la función con el parámetro n.	f(5) en Call Stack
2. Evaluación	Bifurcación recursiva en f(n-1) y f(n-2).	Subárbol de cálculos
3. Transformación	Verificación en caché: si el resultado ya existe, lo devuelve inmediatamente sin recalcular.	Hit en caché O(1)
4. Retorno / Salida	Si no existe, computa el caso base y almacena el resultado antes de retornar.	Guardado en memoria

Visualización Mental

La memoización es intercambiar memoria (RAM) por tiempo de CPU: un compromiso altamente beneficioso en sistemas modernos.

Fibonacci Optimizado con Memoización

main.py (Python 3.10+)

PEP 8 Compliant

```
from functools import lru_cache
import time

# Versión Optimizada con Programación Dinámica
@lru_cache(maxsize=None)
def fibonacci_memo(n: int) -> int:
    if n <= 0: return 0
    if n == 1: return 1
    return fibonacci_memo(n - 1) + fibonacci_memo(n - 2)

# Cálculo instantáneo para n=100
inicio = time.perf_counter()
resultado = fibonacci_memo(100)
fin = time.perf_counter()

print(f"Fibonacci(100) = {resultado}")
print(f"Tiempo de cálculo: {(fin - inicio)*1000:.4f} ms")
```

⚠ Gotcha Frecuente (Trampa de Principiante)

Olvidar el caso base o no avanzar hacia él en cada iteración, provocando un RecursionError por desbordamiento de pila.

✗ Antipatrón

```
def loop(n):
    return loop(n) # RecursionError: maximum
recursion depth exceeded
```

✓ Patrón Correcto

```
def loop(n):
    if n <= 0: return 0 # Caso base
    return n + loop(n - 1)
```

Conclusiones Generales del Curso

Has completado el recorrido integral de **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. El dominio de estos conceptos te sitúa con una ventaja competitiva sólida para el desarrollo de software profesional e Inteligencia Artificial.

Hitos Alcanzados

Comprensión profunda de la arquitectura, asimilación de patrones de diseño idiomáticos en Python, resolución de problemas algorítmicos y capacidad de integración de proyectos reales.

Fuentes Bibliográficas Canónicas

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica del lenguaje	docs.python.org/3/
PEP 8 - Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones profesionales de ingeniería	realpython.com
Suite wisrovi en GitHub	Librerías open source de alto rendimiento	github.com/wisrovi

Agradecimiento del Instructor

Gracias por tu disciplina, curiosidad y compromiso en cada lección. Continúa pedaleando y construyendo software con propósito. ¡Nos vemos en el siguiente nivel!